

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31
- 32
- 33
- 34
- 35
- 36
- 37
- 38
- 39
- 40
- 41
- 42
- 43
- 44
- 45
- 46
- 47
- 48
- 49

5
6

7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

22

23

25

Anonymous Author(s). 2026. A Specification-Driven Multi-Agent Framework for LLM-Based Code Generation
and Validation via Functional Scenario Form. 1, 1 (January 2026), 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

30

Large language models (LLMs) have recently demonstrated strong capabilities in automatic code generation [48]. They can generate syntactically correct programs and often solve programming tasks with impressive performance. However, LLM-generated code is not always reliable and may contain hallucinations [30], logical errors [4], or incorrect handling of corner cases [40]. As a result, validating the generated code is essential before it can be trusted in practice [31].

Testing is a widely used technique for validating programs. However, testing is not only about
executing programs with test inputs [6]; a central challenge lies in defining test oracles [5], i.e.,
determining whether the observed outputs meet the expected behavior. In many existing LLM-
based and multi-agent code generation approaches [17, 19, 20, 39], test inputs can be generated
automatically, but the expected behaviors are often heuristic, incomplete, or inferred from the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/1-ART

48 <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

implementation itself [37]. Without reliable test oracles, testing cannot effectively determine whether the generated code behaves as intended [21].

To obtain reliable test oracles, the intended behavior of a program must be clearly defined. Specification-based testing [38, 42, 46] addresses this need by deriving test cases and oracles directly from a program's specification. However, informal or ambiguous specifications may allow multiple interpretations, which can lead to inconsistent test oracles. Formal specifications address this limitation by providing precise and unambiguous semantics, enabling the expected outcomes of test executions to be determined systematically rather than inferred or guessed [35]. As a result, formal specification-based testing has long been regarded as a reliable approach to software validation [24].

Despite its advantages, formal specification-based testing is rarely used in LLM-based code generation. One main reason is that writing formal specifications is difficult and expensive [13]. Many formal specification languages are complex and require substantial expertise [45], which makes them impractical for most users and hard even for LLMs to produce correctly. As a result, the use of formal specifications in LLM-based code generation remains limited in practice.

To address this challenge, we introduce *FSF* (Functional Scenario Form) [33], a lightweight and template-based formal specification derived from the Structured Object-oriented Formal Language (SOFL) [32]. FSF represents program behavior as a disjunction of functional scenarios. Each functional scenario is defined by a test condition T and a defining condition D . The test condition T specifies the condition under which the scenario applies and constrains the valid input space of the process. The defining condition D characterizes the required relationship between input and output variables, and defines the expected output behavior when T holds. Compared with traditional formal specification languages, FSF is simpler and easier to write, while still providing precise behavioral descriptions. Its structured, scenario-based form supports incremental and step-by-step construction. With the assistance of LLMs, users can transform natural language requirements into FSF specifications by filling in predefined templates, without requiring deep expertise in formal specification languages.

Based on FSF, we propose a specification-driven multi-agent framework for code generation and validation. In this framework, an agent refers to a large language model that can autonomously perform a specific task. The proposed framework coordinates multiple task-specific agents in the entire pipeline. Using FSF with LLMs naturally involves multiple steps, such as refining specifications, generating code, testing the generated programs, and debugging based on test results. When a single agent performs all these steps, it tends to maintain self-consistency [28], which may cause early mistakes to be carried over and reinforced in later stages. To mitigate this issue, we adopt a multi-agent design in which one agent assists users in translating natural-language requirements into FSF specifications, while other agents focus on code generation, test and oracle derivation, and debugging. FSF serves as a shared lightweight formal specification across agents, providing a common reference throughout the workflow.

This paper makes the following contributions:

- We introduce FSF as a template-based formal specification that enables LLMs to transform natural language requirements into formal specifications.
- We demonstrate that FSF supports reliable test and oracle generation for validating LLM-generated code within a specification-driven multi-agent framework.

2 Preliminaries

2.1 Functional Scenario Form (FSF)

Functional Scenario Form is a way to specify the behavior of a process by explicitly listing its functional scenarios. Each functional scenario describes one possible behavior of the process under a specific condition. An FSF specification defines the overall behavior of a process as a disjunction of such scenarios. Formally, an FSF specification has the following form:

$$\text{FSF} \triangleq (T_1 \wedge D_1) \vee (T_2 \wedge D_2) \vee \dots \vee (T_n \wedge D_n),$$

where each pair (T_i, D_i) defines one functional scenario. The test condition T_i specifies the input conditions under which the scenario applies. It is a logical condition over the input variables (and external variables with their initial values) and does not involve any output variables. The defining condition D_i specifies the expected behavior of the process under T_i and describes the required relationship between input and output variables.

The meaning of a functional scenario is simple. If the input of the process satisfies the test condition T_i , then the output produced by the process must satisfy the defining condition D_i . Each functional scenario therefore defines an independent sub-function of the process, and the disjunction of all scenarios specifies the complete behavior of the process.

Example. Consider the following natural language requirement for a customer registration function:

If a customer does not exist in the customer list, the system shall add the customer to the list and return true. Otherwise, if the customer already exists, the system shall leave the list unchanged and return false.

We illustrate FSF using this simple customer registration function.

Semi-formal FSF

Following the FSF style, the functional behavior is described as follows:

```
process RegisterCustomer(customer: Customer) res: bool
ext wr customer_list: seq of Customer

FSF:
The input customer does not exist in the customer list &&
the customer list contains the customer and the result is true
||
The input customer exists in the customer list &&
the customer list is the same as the original customer list and the result is false
end_process;
```

Formal FSF

The above semi-formal FSF can be rewritten into the following formal representation:

$$\begin{aligned} \text{FSF} &= (T_1 \wedge D_1) \vee (T_2 \wedge D_2) \\ T_1 &: \text{customer} \notin \text{customer_list} \\ D_1 &: \text{customer_list}' = \text{customer_list} \cup \{\text{customer}\} \wedge \text{res} = \text{true} \\ T_2 &: \text{customer} \in \text{customer_list} \\ D_2 &: \text{customer_list}' = \text{customer_list} \wedge \text{res} = \text{false} \end{aligned}$$

More details on FSF can be found in the Agile-SOFL book [33].

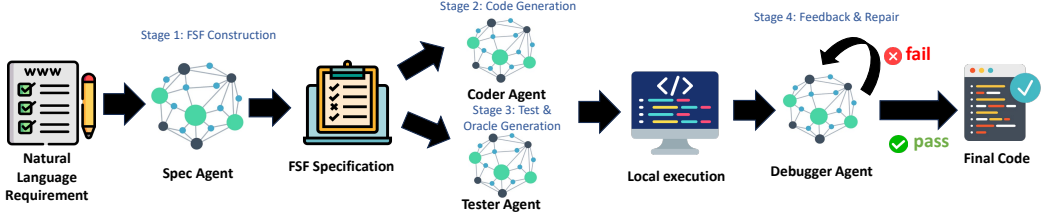


Fig. 1. Overview of the FSF-based multi-agent workflow for specification-guided code generation, testing, and repair.

3 Methodology

This section presents the methodology of our approach. Section 2.1 provides an overview of the multi-agent framework, including its main components and workflow. Sections 2.2–2.5 then describe the four agents in detail, namely the Spec Agent, the Coder Agent, the Tester Agent, and the Debugger Agent.

3.1 Overview

Fig. 1 illustrates the overall workflow of our framework. The framework coordinates four LLM-based agents, namely the Spec Agent, the Coder Agent, the Tester Agent, and the Debugger Agent, to transform natural language requirements into validated code. The workflow is organized into four stages, each corresponding to the responsibility of one agent, and is centered around a shared FSF specification that serves as the semantic reference throughout the process.

Stage 1 – FSF Specification Construction. The process starts with the Spec Agent, which helps users translate a natural language task description into a specification in the Functional Scenario Form (FSF). FSF describes program behavior as a disjunction of functional scenarios, each defined by a test condition T and a defining condition D . The Spec Agent provides a predefined FSF template and assists users in filling in and revising the template according to the given requirements. Through this process, users gradually refine the filled templates into a lightweight formal specification that captures the expected input–output behaviors of the program. The resulting FSF specification is checked for consistency and shared across agents as a common semantic reference.

Stage 2 – Specification-Guided Code Generation. Guided by the FSF specification, the Coder Agent generates a candidate program implementation that aims to satisfy the defining conditions of all functional scenarios. The FSF specification constrains the intended behavior of the program and provides semantic guidance during code generation. Based on feedback from later stages, the Coder Agent may iteratively revise the implementation to better align with the specification.

Stage 3 – Test and Oracle Generation. The Tester Agent generates executable test cases and corresponding test oracles from the FSF specification. For each functional scenario, the test condition T is used to construct valid test inputs under which the scenario applies. Given a test input satisfying T , the defining condition D specifies the expected input–output relationship and is used as the oracle for checking the program output. By deriving both test inputs and oracle conditions from the same FSF specification, this stage ensures that the generated tests and validation results are consistent with the intended specification semantics.

Stage 4 – Execution, Feedback, and Repair. The generated code is executed in a local environment against the FSF-derived test cases, and the execution results are collected. Based on the observed test outcomes, the Debugger Agent analyzes the results and reports violated scenario

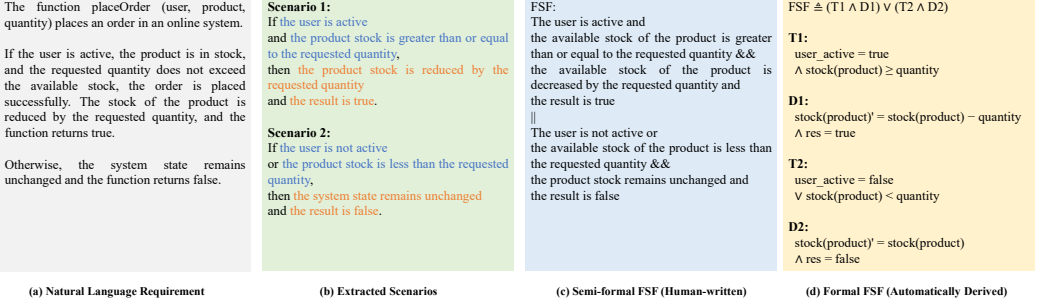


Fig. 2. FSF construction process for placeOrder: (a) natural language requirement, (b) extracted scenarios, (c) semi-formal FSF, and (d) formal FSF.

conditions, indicating potential inconsistencies between the code and the specification. This feedback is then passed back to the Coder Agent, enabling specification-guided debugging and iterative refinement. This process continues until the generated code satisfies all FSF-based tests.

Through these coordinated stages, the proposed framework integrates specification construction, code generation, test and oracle derivation, and debugging into a unified workflow. By using FSF as a shared specification across agents, the framework enables test and oracle generation and supports specification-based validation for LLM-generated code.

3.2 Spec Agent

The Spec Agent assists users in constructing FSF specifications from natural language requirements using a template-based and human-in-the-loop approach. The key idea is to decompose a requirement into functional scenarios and to guide users to describe each scenario using a fixed FSF template. This design keeps the specification process simple for users while preserving the structure required for validation.

3.2.1 FSF Template Design. The FSF template is designed to capture the information required to define a functional scenario. Each functional scenario corresponds to a single template instance. This template instance consists of a scenario identifier, a brief scenario description, a test condition, and a defining condition. The scenario description is only used to support human understanding and is not part of the formal specification. The test condition describes the input conditions under which the scenario applies and is restricted to input variables and initial external states. The defining condition describes the expected behavior when the test condition holds and specifies outputs or state changes. This separation directly follows the definition of FSF and ensures that test conditions and defining conditions play distinct roles.

3.2.2 Specification Construction Process. To make the specification construction process easier to understand, we use a practical example from a real-world development scenario, placeOrder, to illustrate how each agent operates. Given a natural language requirement (Fig. 2), the Spec Agent follows a step-by-step process to construct an FSF specification.

First, the Spec Agent analyzes the natural language requirement and highlights key condition and phrases to identify candidate functional scenarios (Fig. 2(a)). This step is purely supportive and does not determine the final semantics. Second, the Spec Agent suggests candidate functional scenarios that describe independent system behaviors under different conditions (Fig. 2(b)). Each scenario is expressed as a short natural language statement and reviewed by users, who may merge,

revise, or remove scenarios before confirming the final set. Third, for each confirmed scenario, the Spec Agent instantiates an FSF template and guides users to describe the scenario in a semi-formal form, including the test condition and the defining condition (Fig. 2(c)). During template filling, the Spec Agent uses an LLM to provide assistance, such as suggesting candidate conditions, clarifying template fields, and checking for obvious issues. The LLM does not decide the final content of the specification or introduce new assumptions. Finally, once the semi-formal FSF is confirmed by users, the Spec Agent automatically refines it into a formal FSF specification by translating test conditions and defining conditions into logical predicates (Fig. 2(d)). This refinement preserves the original scenario structure and semantics. The resulting formal FSF specification is then used by other agents for code generation, test and oracle construction.

Detailed FSF templates and prompt designs are provided in Appendix A.

3.2.3 Specification Review and Confirmation. Once a formal FSF specification is generated, users are expected to review and confirm its accuracy before it is used by other agents. Although the Spec Agent assists in constructing and refining the specification, it does not guarantee that the specification is complete or accurate. In practice, the generated specification can be checked through manual inspection or existing automated techniques, such as consistency checking [10] and specification animation [26, 34]. Only after the specification is confirmed should it be used for subsequent code generation, test generation, and debugging.

3.3 Coder Agent

The Coder Agent generates and revises program implementations under the guidance of an FSF specification. Its goal is to generate code that satisfies the defining conditions of all functional scenarios. Given a formal FSF specification, the Coder Agent treats each functional scenario as a behavioral requirement.

This agent first uses a large language model to generate an initial implementation from the task description and the FSF specification. This initial implementation is treated as a candidate solution and may not fully satisfy the specification. In particular, although the code is generated according to the FSF, it may still violate some defining conditions on certain execution paths. For example, as shown in Fig. 3, the operation should fail, but the system state is still changed: the inventory is reserved at line 45 before the user state is checked, and the change is not rolled back when the method returns failure at lines 51–52. When violations are identified in later stages, the Coder Agent revises the implementation accordingly. The revision focuses on fixing the behaviors associated with the violated functional scenarios, while leaving other behaviors unchanged.

By grounding both code generation and revision in the FSF specification, the Coder Agent keeps the implementation aligned with the intended specification throughout the workflow.

3.4 Tester Agent

The Tester Agent validates candidate implementations using specification-based testing. As shown in Fig. 4, both test cases and test oracles are derived directly from the same FSF specification. Each functional scenario in the FSF contributes test cases to the overall test set. In this way, testing follows the specification rather than the implementation.

3.4.1 Test Case Generation. Test cases are generated based on the test conditions defined in the FSF specification. For each functional scenario (T_i, D_i) , the Tester Agent creates test inputs that satisfy the test condition T_i . These inputs are used to execute the program under the situation described by the scenario. As illustrated in Fig. 4, the complete test suite is formed by combining the test cases generated for all functional scenarios. Each scenario is tested independently, and test cases are derived only from the specification.


```

1  from dataclasses import dataclass
2
3  @dataclass
4  class User:
5      active: bool
6
7  @dataclass
8  class Product:
9      stock: int
10
11 class InventoryService:
12     """
13     Inventory operations. Reserve mutates stock immediately (side
14     effect).
15     """
16
17     def reserve(self, product: Product, quantity: int) -> bool:
18         if quantity <= 0:
19             return False
20         if product.stock < quantity:
21             return False
22         product.stock -= quantity # reserve by decrementing
23         stock
24         return True
25
26     def release(self, product: Product, quantity: int) -> None:
27         product.stock += quantity
28
29 class OrderServiceBuggy:
30     def __init__(self) -> None:
31         self.inventory = InventoryService()
32
33     def place_order(self, user: User, product: Product, quantity:
34     int) -> bool:
35         """
36         Intended spec:
37         - If user is active and stock >= quantity: stock' = stock
38         - quantity, res=True
39         - Else: stock unchanged, res=False
40         """
41
42         # Engineering-style pattern: reserve first to reduce race
43         windows.
44         reserved = self.inventory.reserve(product, quantity)
45
46         # BUG (non-trivial): user may be inactive. In that case
47         the order must fail
48         # with no state changes, but the reservation is not
49         rolled back.
50         if not user.active:
51             return False # violates D2 when reserve() already
52             decremented stock
53
54         # If reservation failed, the order fails.
55         if not reserved:
56             return False
57
58         # Success path: reservation already decremented stock.
59         return True
60

```

Fig. 3. Initial implementation of placeOrder.

3.4.2 Oracle Generation. For each test case, the defining condition D_i of the corresponding functional scenario is used as the test oracle. After executing the program with a test input satisfying T_i , the observed result is checked against D_i . The test passes if the defining condition is satisfied. Otherwise, the test fails. As shown in Fig. 4, both the test case and its oracle come from the same functional scenario. This keeps the test results consistent with the FSF specification.

The test cases shown in Fig. 4 are only a small subset used to illustrate the generation process. In practice, the Tester Agent can generate many more test cases from the FSF specification to cover different functional scenarios and their possible execution paths. More details can be found in [35].

3.5 Debugger Agent

The Debugger Agent analyzes execution results and supports specification-guided debugging and repair. As shown in Fig. 5, it explains test failures using the FSF specification and connects testing results with code revision.

In our example, the initial implementation fails the FSF-based tests because it updates the system state when the operation should fail. According to the FSF specification, if the user is inactive or the stock is insufficient, the order should fail and the system state should remain unchanged. However, in the initial implementation, the stock is reduced before all failure conditions are checked. As a

```

1 import unittest
2 # Assume User, Product, OrderServiceBuggy are already defined above.
3
4 class FSFOracles:
5     @staticmethod
6     def oracle_D1_success(stock_before: int, stock_after: int, res: bool, quantity: int) -> None:
7         # D1: stock' = stock - quantity A res = true
8         assert res is True, "D1: res must be true"
9         assert stock_after == stock_before - quantity, \
10             f"D1: stock must decrease by quantity (before={stock_before}, after={stock_after}, \
11              quantity={quantity})"
12
13     @staticmethod
14     def oracle_D2_failure(stock_before: int, stock_after: int, res: bool) -> None:
15         # D2: stock' = stock A res = false
16         assert res is False, "D2: res must be false"
17         assert stock_after == stock_before, \
18             f"D2: stock must remain unchanged (before={stock_before}, after={stock_after})"
19
20 class TestPlaceOrderSpec(unittest.TestCase, FSFOracles):
21
22     def setUp(self) -> None:
23         self.svc = OrderServiceBuggy()
24
25     # ----- Scenario 1 (T1, D1) : Success functional scenarios -----
26
27     def test_T1_success_stock_sufficient(self):
28         # T1: user_active=True A stock>=quantity
29         user = User(active=True)
30         product = Product(stock=10)
31         quantity = 4
32
33         before = product.stock
34         res = self.svc.place_order(user, product, quantity)
35         after = product.stock
36         self.oracle_D1_success(before, after, res, quantity)
37
38     def test_T1_boundary_stock_equals_quantity(self):
39         # Boundary for T1: stock == quantity still satisfies stock>=quantity
40         user = User(active=True)
41         product = Product(stock=4)
42         quantity = 4
43
44         before = product.stock
45         res = self.svc.place_order(user, product, quantity)
46         after = product.stock
47         self.oracle_D1_success(before, after, res, quantity)
48
49     # ----- Scenario 2 (T2, D2) : Failure functional scenario -----
50
51     def test_T2_failure_insufficient_stock(self):
52         # T2 via stock < quantity
53         user = User(active=True)
54         product = Product(stock=3)
55         quantity = 4
56
57         before = product.stock
58         res = self.svc.place_order(user, product, quantity)
59         after = product.stock
60         self.oracle_D2_failure(before, after, res)
61
62     def test_T2_failure_inactive_user_state_unchanged(self):
63         # T2 via user_active=False (stock is sufficient but user inactive)
64         # Expected by D2: stock unchanged A res=False
65         # This test fails for the buggy implementation due to state leak.
66         user = User(active=False)
67         product = Product(stock=10)
68         quantity = 4
69
70         before = product.stock
71         res = self.svc.place_order(user, product, quantity)
72         after = product.stock
73         self.oracle_D2_failure(before, after, res) # expected to fail (after becomes 6)
74
75 if __name__ == "__main__":
76     unittest.main()

```

Fig. 4. Example FSF-derived test cases and oracles for placeOrder.

result, the test for the failure scenario detects an unexpected state change, and the oracle reports a failure.

The Debugger Agent reports which functional scenario is violated and which defining condition is not satisfied. Based on this feedback, the required fix is clear: all failure conditions must be checked before any state update. Guided by this feedback, the Coder Agent revises the implementation so that state changes occur only when the success scenario holds. As shown in Fig. 5, the revision moves all failure checks (lines 13–18) before the state update (line 21), ensuring that the defining condition of the failure scenario is satisfied.

This execute–feedback–repair process is repeated until the implementation passes all FSF-based tests. In this way, the Debugger Agent helps improve LLM-generated code using specification-level feedback.

4 Experimental Design

4.1 Research Questions

This study evaluates our framework by addressing three research questions:

- **RQ1.** How well does the framework perform in generating code?
- **RQ2.** How does each core component contribute to the framework’s performance?
- **RQ3.** What computational costs are incurred when running the framework?


```

1  class OrderServiceFixed:
2      def __init__(self) -> None:
3          self.inventory = InventoryService()
4
5      def place_order(self, user: User, product: Product, quantity: int) -> bool:
6          """
7          Spec-compliant implementation:
8          - No state change on failure (D2)
9          - Stock decreases only on success (D1)
10         """
11
12         # Fail-fast checks ensure no side effects when the order fails.
13         if not user.active:
14             return False
15         if quantity <= 0:
16             return False
17         if product.stock < quantity:
18             return False
19
20         # Now T1 holds; safe to mutate state.
21         reserved = self.inventory.reserve(product, quantity)
22         if not reserved:
23             return False
24
25         return True

```

Fig. 5. Repaired implementation of placeOrder guided by FSF.

4.2 Datasets

Following the SOTA method CodeSIM, we evaluate our framework on five basic programming benchmarks, **HumanEval** [9], **HumanEval-ET** [11], **EvalPlus** [31], **MBPP** [3], and **MBPP-ET** [11], as well as two competitive programming datasets: **APPS** [15] and **CodeContest** [27].

4.3 Baselines

To assess the performance of our framework, we compare it with several state-of-the-art code generation methods, including CodeSIM [20], CodeCoR [39], CodeCoT [17], and MapCoder [19]. We also consider widely used baselines such as Direct prompting, Chain of Thought (CoT) [51], Self-Planning [22], Analogical Reasoning [52], and Reflexion [44].

Fairness Consideration. Our framework assumes a lightweight formal specification (FSF) as a core design choice. The baselines do not use formal specifications and are evaluated with natural language inputs under their original settings. This reflects a realistic setting in which different methods use different representations. To further ensure fairness, we perform ablation studies (RQ2) by removing the FSF component from our framework to isolate its impact.

4.4 Base Large Language Models

For fairness, we evaluate our framework using both proprietary and open-source LLMs. From OpenAI, we include **GPT-4** (gpt-4-1106-preview) and **GPT-4o** (gpt-4o-2024-08-06). We also test several strong open-source models, including **Gemma2-9B**, **Mixtral8x7B**, **LLaMa3.1-8B**, and **LLaMa3.1-70B**. For basic programming benchmarks, we additionally report results from the next-generation **GPT-5-codex** (gpt-5-2025-09-15).

Table 1. Pass@1 results for different approaches on **basic programming tasks**. Bold numbers indicate the best results in each column. All improvements of our framework over CodeSim are statistically significant (paired t -test, $p < 0.01$).

LLM	Approach	HumanEval	HumanEval ET	EvalPlus	Avg HumanEval	MBPP	MBPP-ET	Avg MBPP	Avg
GPT4	Direct	80.1%	73.8%	81.7%	78.5%	81.1%	54.7%	67.9%	74.3%
	CoT	89.0%	61.6%	79.8%	76.8%	82.6%	56.2%	69.4%	73.8%
	Self-Planning	85.4%	62.2%	78.4%	75.3%	75.8%	50.4%	63.1%	70.4%
	Analogical	66.5%	48.8%	62.2%	59.1%	58.4%	40.3%	49.4%	55.2%
	Reflexion	91.0%	78.7%	81.7%	83.8%	78.3%	51.9%	65.1%	76.3%
	CodeCoT	91.5%	80.3%	82.4%	84.7%	80.7%	56.8%	68.8%	78.3%
	MapCoder	93.9%	82.9%	83.5%	86.8%	83.1%	57.7%	70.4%	80.2%
	CodeCoR	94.1%	83.5%	84.1%	87.2%	86.1%	60.2%	73.2%	81.6%
	CodeSim	94.5%	81.7%	84.8%	87.0%	89.7%	61.5%	75.6%	82.4%
	Our framework	96.2%	85.4%	88.1%	89.9%	91.0%	63.9%	77.5%	84.9%
GPT4o	Direct	90.2%	81.1%	82.3%	84.5%	81.1%	55.9%	68.5%	78.1%
	CoT	90.9%	82.3%	87.2%	86.8%	82.9%	57.9%	70.4%	80.24%
	Self-Planning	89.0%	80.5%	84.1%	84.5%	82.6%	56.4%	69.5%	78.5%
	Analogical	88.4%	80.5%	83.5%	84.1%	75.1%	50.9%	63.0%	75.6%
	Reflexion	87.2%	81.1%	81.1%	83.1%	81.1%	56.7%	68.9%	77.4%
	CodeCoT	89.7%	81.8%	85.4%	85.6%	87.2%	58.8%	73.0%	80.6%
	MapCoder	90.2%	80.5%	81.7%	84.1%	88.7%	59.2%	74.0%	80.0%
	CodeCoR	92.7%	83.6%	86.2%	87.5%	89.5%	60.8%	75.2%	82.5%
	CodeSim	95.1%	86.0%	87.2%	89.4%	90.7%	61.2%	76.0%	84.0%
	Our framework	97.3%	88.5%	89.6%	91.8%	93.2%	65.4%	79.3%	86.8%
GPT5	Direct	92.8%	84.0%	86.3%	87.7%	84.2%	58.6%	71.4%	81.2%
	CoT	93.7%	85.2%	89.3%	89.4%	86.5%	60.7%	73.6%	83.0%
	Self-Planning	91.9%	83.6%	87.1%	87.5%	85.7%	59.2%	72.5%	81.5%
	Analogical	90.4%	82.1%	85.4%	86.0%	79.8%	54.1%	67.0%	78.4%
	Reflexion	91.5%	83.4%	86.5%	87.1%	84.0%	58.0%	71.0%	80.7%
	CodeCoT	92.5%	84.6%	88.3%	88.5%	89.1%	60.4%	74.8%	83.0%
	MapCoder	93.0%	85.4%	88.7%	89.0%	90.2%	61.0%	75.6%	83.6%
	CodeCoR	95.2%	87.3%	89.9%	90.8%	91.3%	62.4%	76.9%	85.2%
	CodeSim	96.8%	88.4%	91.1%	92.1%	92.8%	64.0%	78.4%	86.6%
	Our framework	98.5%	90.3%	93.0%	93.9%	95.0%	66.8%	80.9%	88.7%

4.5 Metric

We use the standard $pass@1$ [9] metric. For each task, the model generates one solution. The task is considered solved if this solution passes all test cases and test oracles. The test cases used for evaluation are provided by the benchmark datasets and are not generated by our method. This metric reflects a practical setting in which the model is expected to produce a single usable solution without repeated attempts.

4.6 Implementation Details

We implemented our framework in Python and executed all experiments locally in a controlled environment. Test cases and test oracles derived from FSF were implemented using Python’s unittest framework.

To ensure reproducibility and control computational cost, all prompts and execution settings were fixed, and at most one repair attempt was allowed for each task. If the code still failed the FSF-based tests after repair, the task was considered unsuccessful.

5 Experimental Results

5.1 RQ1. How effective is our framework in generating code? (Performance Evaluation)

5.1.1 Basic Code Generation. Table 1 presents the results on basic code generation tasks. Our framework consistently outperforms all baselines across every dataset and LLM. With GPT-5, it attains 98.5% on HumanEval, 93.0% on EvalPlus, and 95.0% on MBPP, yielding an overall average

Table 2. Pass@1 results (%) on **contest-level problems**. All improvements of our framework over CodeSim are statistically significant ($p < 0.01$).

LLM	Approach	CodeContest	APPS	Avg
GPT4	Direct	12.1	12.7	12.4
	CoT	5.5	11.3	8.4
	Self-Planning	10.9	14.7	12.8
	Analogical	10.9	12.0	11.5
	CodeCoT	29.8	22.7	26.3
	MapCoder	28.5	22.0	25.3
	CodeCoR	30.5	23.4	26.9
	CodeSim	29.1	22.0	25.6
	Our framework	32.8	24.9	28.9
GPT4o	Direct	13.2	13.8	13.5
	CoT	6.7	12.6	9.6
	Self-Planning	12.1	15.9	14.0
	Analogical	12.2	13.1	12.6
	CodeCoT	31.9	24.5	28.2
	MapCoder	30.6	23.9	27.3
	CodeCoR	33.1	25.5	29.3
	CodeSim	32.0	24.2	28.1
	Our framework	35.2	26.8	31.0

of 88.7%. This surpasses the next-best method, CodeSim, which achieves 86.6% on average using GPT-5. Our framework also shows strong performance with other model families, reaching 84.9% with GPT-4 and 86.8% with GPT-4o. Its robustness extends to more challenging benchmarks such as MBPP-ET, where it achieves 66.8% with GPT-5. Overall, these results highlight our framework’s ability to deliver high-quality code generation across diverse tasks and model scales, demonstrating clear improvements over existing approaches.

5.1.2 Competitive Problem Solving. Table 2 reports results on complex, contest-level code generation tasks. Our framework shows clear improvements over all baselines in this setting. With GPT-4o, it achieves 35.2% on CodeContests, 26.8% on APPS and 31.0% on average, outperforming CodeSim’s 28.1% average. The advantage is similar with GPT-4, where our framework reaches 32.8% on CodeContests and 24.9% on APPS, yielding a 28.9% overall—higher than CodeSim’s 25.6%. These results demonstrate that our framework is more capable of addressing the complexity and reasoning demands of contest-level programming problems.

5.1.3 Performance Across Open-source LLMs. To assess our framework’s generalization ability, we further test it on several open-source LLMs, including Gemma2-9B, Mixtral-8×7B, LLaMA3.1-8B, and LLaMA3.1-70B. As shown in Table 3, our framework consistently outperforms all baselines on these models. With LLaMA3.1-70B, it reaches 92.4% on HumanEval and 78.0% on EvalPlus, achieving an average accuracy of 81.9%. Because MapCoder requires a highly intricate prompting format that open-source models frequently fail to follow, we omit it from this evaluation. Even on smaller models such as Gemma2-9B, our framework maintains strong performance, achieving a 78.6% average. These results demonstrate that our framework generalizes well across a wide range of LLM architectures.

Table 3. Pass@1 results (%) on **open-source LLMs**. All improvements of our framework over CodeSim are statistically significant ($p < 0.01$).

LLM	Approach	HumanEval	HumanEval ET	EvalPlus	Avg
Gemma2-9B	Direct	64.0	56.1	56.1	58.7
	CoT	31.7	26.2	27.4	28.4
	Reflexion	62.2	56.7	55.5	58.1
	CodeCoT	78.3	68.9	69.4	72.2
	CodeCoR	84.0	71.0	72.0	75.7
	CodeSim	82.9	72.0	72.6	75.8
	Our framework	86.5	74.3	75.0	78.6
Mixtral-8×7B	Direct	20.7	18.9	18.9	19.5
	CoT	46.3	42.3	39.0	42.5
	Reflexion	34.1	33.1	29.9	32.3
	CodeCoT	70.8	59.3	61.0	63.7
	CodeCoR	74.3	62.2	63.8	66.8
	CodeSim	75.0	61.6	61.6	66.0
	Our framework	77.8	64.2	65.8	69.3
LLaMA3-8B	Direct	42.1	38.4	39.0	39.8
	CoT	48.8	42.1	43.3	44.7
	Reflexion	43.9	31.1	29.9	33.0
	CodeCoT	76.2	62.3	63.7	67.4
	CodeCoR	78.6	64.0	65.9	69.5
	CodeSim	79.9	65.2	61.9	69.0
	Our framework	82.4	67.8	68.5	72.9
LLaMA3-70B	Direct	57.3	50.6	52.4	53.4
	CoT	75.6	67.7	70.1	71.1
	Reflexion	73.8	64.0	68.8	68.9
	CodeCoT	88.3	72.8	75.1	78.7
	CodeCoR	89.2	73.1	75.6	79.3
	CodeSim	90.2	73.8	76.2	80.1
	Our framework	92.4	75.4	78.0	81.9

Answer to RQ1: Our framework outperforms existing LLM-based code generation methods across multiple benchmarks.

5.2 RQ2. How does each major component contribute to the overall performance of our framework? (Ablation Study)

This section examines how each core component contributes to the effectiveness of our framework. We compare the full system with two variants:

- **NL-Agent (no FSF formal language).** This variant keeps the same multi-agent structure but removes the Spec Agent. Natural language requirements are not converted into formal specifications; instead, the Coder, Tester, and Debugger Agents interact purely through natural language.

Table 4. The impact of major components of our framework on HumanEval and MBPP datasets (Pass@1). All improvements of our framework over other variants are statistically significant ($p < 0.01$).

Variant	HumanEval	HumanEval-ET	MBPP	MBPP-ET
NL-Agent	85.3	77.5	80.2	59.1
FSF-Coder only	92.6	85.8	88.4	63.7
Our framework	98.5	90.3	95.0	66.8

Table 5. Cost comparison of code generation models.

Method	Run Time (s)	CPU Usage (%)	Memory Usage (GB)	Disk Read (MB)	Disk Write (MB)	Net Send / Receive (MB)
Our framework	110.52	0.7	0.01	0.32	10.35	0.13 / 0.27
CodeSim	178.33	1.0	0.03	0.54	14.62	0.27 / 0.41
CodeCoR	123.69	0.8	0.01	0.36	11.49	0.14 / 0.30
MapCoder	166.45	0.8	0.02	0.48	12.78	0.25 / 0.36
CodeCoT	210.27	1.1	0.03	0.69	16.93	0.31 / 0.52
Self-Planning	242.92	0.2	0.02	1.02	31.16	0.35 / 0.74
CoT	198.54	0.9	0.02	0.62	15.28	0.29 / 0.46

- **FSF-Coder only (no testing or debugging).** This version removes the Tester and Debugger Agents, leaving only the Spec Agent and the Coder Agent. These two agents translate natural language requirements into formal specifications and generate code accordingly.
- **Our framework (full pipeline).** The complete system, where agents use FSF formal language, generate both code and test cases, and perform testing and debugging.

Table 4 summarizes the ablation results. Each component plays a clear role in our framework’s overall effectiveness. Removing the formal specification layer (**NL-Agent**) leads to a substantial drop in Pass@1 across all datasets (85.3% on HumanEval, 77.5% on HumanEval-ET, 80.2% on MBPP, and 59.1% on MBPP-ET). This shows that formal specifications are essential for generating stable programs. The **FSF-Coder only** variant performs better, reaching 92.6%, 85.8%, 88.4%, and 63.7% respectively. The results confirm that the Spec and Coder Agents can effectively translate requirements into formal specifications and executable code. However, without testing and debugging, the system cannot refine or verify outputs, limiting further accuracy gains. The full pipeline achieves the highest performance on all benchmarks (98.5%, 90.3%, 95.0%, and 66.8%). The addition of collaborative testing and debugging noticeably improves reliability by catching inconsistencies and iteratively refining the generated code. Overall, the findings highlight that both formal specification and multi-agent collaboration are critical for achieving reliable, high-quality code generation.

Answer to RQ2: The ablation study shows that all components of our framework are essential. Removing either the formal specification module or the testing–debugging steps leads to clear accuracy drops, confirming that both elements are crucial for reliable code generation.

5.3 RQ3. What are the computational costs of our framework?

Most multi-agent systems incur higher computational costs than single-agent approaches. To assess this aspect, we compare the cost of running our framework with several representative baselines, including CodeSim, CodeCoR, MapCoder, CodeCoT, CoT, and Self-Planning. Table 5 reports execution time, CPU usage, memory consumption, disk I/O, and network I/O. All experiments were conducted on ten HumanEval tasks in a controlled Python environment, with system resource usage recorded via psutil.

Our framework achieves the lowest overall runtime (110.52 seconds), slightly outperforming CodeCoR (123.69 s) and clearly faster than MapCoder (166.45 s), CodeCoT (210.27 s), and Self-Planning (242.92 s). The framework also maintains very low CPU usage (below 1%), highlighting its efficiency. In memory usage, our framework requires only 0.01 GB, comparable to CodeCoR and substantially smaller than CoT's 0.02 GB. Our framework keeps network usage modest (0.13 MB send / 0.27 MB receive), outperforming CodeSim (0.27/0.41 MB) and MapCoder (0.25/0.36 MB).

Overall, our framework offers the best cost–performance balance among all compared methods. Its efficiency stems from combining formal specifications with multi-agent collaboration, reducing unnecessary computations and communication overhead.

Answer to RQ3: Our framework provides the best cost efficiency among all compared methods, lowering runtime and resource usage while still producing high-quality code.

6 Discussion

6.1 Limitations

Our framework has several limitations, which we discuss below.

Dependence on FSF specifications. Our framework uses FSF specifications as the common reference for code generation, test generation, and debugging, enabling consistent interpretation across these activities. As with other specification-driven approaches, the quality of the generated artifacts depends on the accuracy and completeness of the FSF specification. Inaccurate or incomplete specifications may therefore lead to unexpected results. Improving or validating specification quality is beyond the scope of this work.

Incomplete natural-language requirements. In practice, natural-language requirements are often short or incomplete, which makes it difficult to construct precise formal specifications. Although our framework supports interactive and template-based specification construction, the initial specification may still be incomplete. Users may therefore need to manually refine it.

Scope of tasks. Our evaluation focuses on function-level tasks, where program behavior can be described with FSF specifications and checked against executable code. In contrast to system-level modeling approaches that focus on abstract system behavior, our work targets the reliability of program implementations.

6.2 Data Leakage Evaluation

A key challenge in evaluating LLM-based code generation is data leakage, where benchmark problems may appear in the model's pre-training data. To mitigate this risk and assess our framework's performance on genuinely unseen tasks, we constructed JavaEval-500—a set of 500 original, manually designed Java function-level problems with corresponding JUnit tests. None of these tasks appear in public code repositories. A brief summary of the dataset is shown in Table 6.

As shown in Table 7, our framework performs strongly on the contamination-free JavaEval benchmark, while the performance of several baselines drops noticeably. This consistent advantage indicates that our framework's gains stem from formal specifications rather than memorization of pre-training data.

6.3 Threats to Validity

We discuss three factors that may influence the validity of our study:

Table 6. Summary of the JavaEval-500 Benchmark

Category	Statistics
Total Problems	500 Java function-level tasks (Java 8+, no external dependencies)
Average Size	Prompt: 3.7 lines; Solution: 5.2 lines; Tests: 9.5 lines
Evaluation	Executable JUnit4 unit tests with deterministic pass/fail outputs

Table 7. Performance comparison on the JavaEval benchmark. All improvements of our framework over the strongest baseline (e.g., CodeSim) are statistically significant (paired t-test, $p < 0.01$).

LLM	Approach	JavaEval
GPT4	Direct	21.5
	CoT	14.2
	Self-Planning	18.9
	Analogical	17.8
	CodeCoT	42.3
	MapCoder	40.8
	CodeCoR	43.5
	CodeSim	44.1
	Our framework	46.3
GPT4o	Direct	23.1
	CoT	15.7
	Self-Planning	20.4
	Analogical	19.6
	CodeCoT	44.8
	MapCoder	42.7
	CodeCoR	45.9
	CodeSim	46.8
	Our framework	48.5

- **Internal Validity:** To minimize implementation-related risks, we used the official source code of all baseline methods from their GitHub repositories and followed the hyperparameters reported in their papers. Our own implementation was also carefully reviewed for potential errors.
- **External Validity:** External threats mainly concern dataset. We mitigated this by evaluating on widely used benchmarks, HumanEval, HumanEval-ET, EvalPlus, MBPP, MBPP-ET, CodeContest, and APPS, together with our newly created JavaEval-500 dataset. Each experiment was repeated five times, and we report average results to reduce randomness.
- **Construct Validity:** Our evaluation uses the Pass@1 metric, following prior work [20]. This standard metric offers a reliable basis for comparing model performance across different approaches.

7 Related Work

7.1 LLM-based Multi-Agent Code Generation

Recent advances in large language models have led to growing interest in multi-agent code generation frameworks. Instead of relying on a single model invocation, these approaches decompose complex programming tasks into multiple stages and assign them to cooperating agents, such as planners, coders, and reviewers. This collaborative setting enables iterative refinement and improves robustness compared to single-pass generation.

At both the function and system levels, representative frameworks such as CodeChain [23], FlowGen [29], CodeTree [25], MapCoder [19], DARS [1], PairCoder [53], and CodePoRi [41] adopt role-based collaboration, iterative feedback, or cooperative reasoning to enhance code quality. Other systems, including GameGPT [8], SoA [18], CodeAgent [54], and ToolGen [49], further extend this idea by integrating external tools and supporting large multi-module projects.

Although these approaches demonstrate the effectiveness of multi-agent coordination, most of them rely on heuristic interaction. They lack an explicit and shared specification that defines the intended program behavior and provides a reference for coordination, validation, and correction across agents.

7.2 LLM-based Multi-Agent Test Generation

Testing is a critical component of software development, and recent studies have explored the use of LLM-based multi-agent systems to automate test generation and validation. By assigning different agents to test generation, execution, and evaluation, these approaches aim to improve coverage and reduce manual effort.

Frameworks such as TestPilot [43], XUnit Copilot [50], and LogiAgent [55] organize agents to generate test cases and assess program outputs. Other systems, including ACH [14] and MetaGPT [16], adopt feedback-driven strategies to iteratively expand test suites and repair detected errors. These studies show that collaboration among agents can improve the effectiveness of automated testing.

However, most existing test generation frameworks remain largely empirical. Their test oracles are often heuristic, and it is unclear why a test is considered correct or incorrect. As a result, these approaches lack clear criteria for interpreting test results and guiding subsequent repairs.

7.3 Integrating Formal Methods with Large Language Models

Recent work has explored combining formal methods with LLMs to improve software reliability. One line of research uses LLMs to assist formal specification construction from natural-language requirements. Typical approaches translate informal descriptions into postconditions, contracts, or logical constraints, aiming to reduce the cost of writing formal specifications. Examples include generating formal postconditions from natural language [12] and producing verifiable specifications through multi-stage generation [36]. Other studies evaluate the ability of LLMs to generate formal specifications or proofs across different formalisms [7].

Another line of work applies LLMs to support formal verification. In these approaches, LLMs assist proof construction, invariant generation, or model checking. For example, ESBMC-AI [47] integrates LLMs with bounded model checking to detect and repair software vulnerabilities. More recently, several studies [2, 56] focus on system-level formal modeling and verification, where LLMs are used to construct abstract system models from natural-language descriptions and validate them via model checking. These approaches typically focus on system behavior at an abstract level and emphasize verification of global properties rather than executable program implementations.

While these studies show that LLMs can bridge natural language, code, and formal artifacts, most focus on single-agent interactions with formal tools or system-level verification tasks. Formal specifications are often treated as static targets or intermediate artifacts, rather than shared references that guide code generation, testing, and repair across multiple agents, as explored in our work.

8 Conclusion

This paper introduces Functional Scenario Form (FSF), a lightweight and template-based formal specification, together with a multi-agent framework for LLM-based code generation and validation. In this framework, FSF serves as a shared reference that guides code generation and enables the derivation of test cases and test oracles. We evaluated the approach on multiple benchmarks, including our self-constructed JavaEval-500 dataset. Overall, the results suggest that using FSF to structure specifications and testing is a useful direction for validating LLM-generated code.

9 Data Availability

An anonymized replication package containing the implementation, prompts, and experimental scripts used in this study is publicly available for review purposes¹, in compliance with the double-anonymous review policy.

In addition, the approach is evaluated on a newly constructed benchmark dataset, JAVAEVAL-500, which consists of 500 Java programming tasks designed to mitigate data contamination and improve evaluation reliability. The dataset is publicly available².

Appendix

A FSF Templates and Prompt Details

These prompts are used to guide the interaction between users and the Spec Agent. They provide assistance but do not determine the final specification content.

A.1 FSF Template

Each functional scenario is represented by one FSF template instance with the following fields.

- **Scenario ID:** A unique identifier of the scenario.
- **Scenario Description:** A brief natural language description of the scenario. This field is only used to support human understanding and is not part of the formal specification.
- **Test Condition (T_i):** A structured description of the input condition under which the scenario applies. It may include relevant input variables, constraints on inputs, and assumptions on initial external states. Output variables are not allowed in this field.
- **Defining Condition (D_i):** A structured description of the expected behavior when the test condition holds. It specifies expected outputs, relationships between inputs and outputs, and updates to external states if any.

¹<https://anonymous.4open.science/r/FSF-Agents-E657/README.md>

²<https://anonymous.4open.science/r/JavaEval-500-B8D8/README.md>

A.2 Scenario Identification

Prompt A: Scenario Identification

You are assisting a user in constructing a specification using Functional Scenario Form (FSF). Given the following natural language requirement, identify possible functional scenarios. Each scenario should describe one independent behavior of the system under a specific condition. Write each scenario as a short sentence. Do not write any formal logic. Do not decide how many scenarios are correct. The user will review and select the final scenarios.

Requirement: <USER_REQUIREMENT>

A.3 Filling the Test Condition

Prompt B: Filling the Test Condition

You are assisting the user in filling the Test Condition of an FSF scenario.

Scenario description: <SCENARIO_DESCRIPTION>

Describe the input conditions when this scenario applies. Use structured natural language. Only refer to input variables or initial states. Do not refer to any output variables.

The user may revise or rewrite your suggestion.

A.4 Filling the Defining Condition

Prompt C: Filling the Defining Condition

You are assisting the user in filling the Defining Condition of an FSF scenario.

Scenario description: <SCENARIO_DESCRIPTION>

Test condition: <USER_DEFINED_TEST_CONDITION>

Describe the expected behavior when the test condition holds. Use structured natural language. Specify the expected output or state change. Do not introduce new assumptions.

The user may revise or rewrite your suggestion.

A.5 Semi-formal to Formal Refinement

Prompt D: Semi-formal to Formal Refinement

You are given a semi-formal FSF scenario.

Test condition: <USER_DEFINED_TEST_CONDITION>

Defining condition: <USER_DEFINED_DEFINING_CONDITION>

Translate the above into a formal FSF scenario. Preserve the original meaning. Do not introduce new variables or conditions.

Output format:

$$(T_i \wedge D_i)$$

References

- [1] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. Dars: Dynamic action re-sampling to enhance coding agent performance by adaptive tree traversal. *arXiv preprint arXiv:2503.14269* (2025).
- [2] Rishabh Agrawal, Kaushik Tushar Ranade, Aja Khanal, Kalyan Shankar Basu, and Apurva Narayan. [n. d.]. SpecMAS: A Multi-Agent System for Self-Verifying System Generation via Formal Model Checking. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [4] Rishabh Balse, Viraj Kumar, Prajish Prasad, and Jayakrishnan Madathil Warriem. 2023. Evaluating the quality of llm-generated explanations for logical errors in cs1 student programs. In *Proceedings of the 16th annual ACM India compute conference*. 49–54.
- [5] Luciano Baresi and Michal Young. 2001. Test oracles. (2001).
- [6] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2014. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2014), 507–525.
- [7] Jialun Cao, Yaojie Lu, Meiziniu Li, Haoyang Ma, Haokun Li, Mengda He, Cheng Wen, Le Sun, Hongyu Zhang, Shengchao Qin, et al. 2025. From Informal to Formal-Incorporating and Evaluating LLMs on Natural Language Requirements to Verifiable Formal Proofs. *arXiv preprint arXiv:2501.16207* (2025).
- [8] Dake Chen, Hanbin Wang, Yunhao Huo, Yuzhao Li, and Haoyang Zhang. 2023. Gamept: Multi-agent collaborative framework for game development. *arXiv preprint arXiv:2310.08067* (2023).
- [9] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [10] Yuting Chen, Shaoying Liu, and W Eric Wong. 2008. A Review Approach to Detecting Violations of Consistency Between Specification and Program Structures. *International Journal of Software Engineering and Knowledge Engineering* 18, 08 (2008), 1013–1042.
- [11] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. Codescore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.
- [12] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1889–1912.
- [13] Andrew Gravell. 1991. What is a good formal specification?. In *Z User Workshop, Oxford 1990: Proceedings of the Fifth Annual Z User Meeting, Oxford, 17–18 December 1990*. Springer, 137–150.
- [14] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-guided llm-based test generation at meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 180–191.
- [15] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).
- [16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.
- [17] Dong Huang, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Codecot: Tackling code syntax errors in cot reasoning for code generation. *arXiv preprint arXiv:2308.08784* (2023).
- [18] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183* (2024).
- [19] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 4912–4944. doi:10.18653/v1/2024.acl-long.269
- [20] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2025. CodeSim: Multi-Agent Code Generation and Problem Solving through Simulation-Driven Planning and Debugging. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 5113–5139. doi:10.18653/v1/2025.findings-naacl.285
- [21] Gunel Jahangirova. 2017. Oracle problem in software testing. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 444–447.
- [22] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [23] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2023. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. *arXiv preprint arXiv:2310.08992* (2023).
- [24] Pascale Le Gall and Agnes Arnould. 1995. Formal specifications and test: Correctness and oracle. In *Workshop on the Specification of Abstract Data Types*. Springer, 342–358.

- [25] Jierui Li, Hung Le, Yingbo Zhou, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. Codetree: Agent-guided tree search for code generation with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3711–3726.
- [26] Mo Li and Shaoying Liu. 2015. Integrating animation-based inspection into formal design specification construction for reliable software systems. *IEEE transactions on reliability* 65, 1 (2015), 88–106.
- [27] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [28] Xun Liang, Shichao Song, Zifan Zheng, Hanyu Wang, Qingchen Yu, Xunkai Li, Rong-Hua Li, Yi Wang, Zhonghao Wang, Feiyu Xiong, et al. 2024. Internal consistency and self-feedback in large language models: A survey. *arXiv preprint arXiv:2407.14507* (2024).
- [29] Feng Lin, Dong Jae Kim, et al. 2024. Soen-101: Code generation by emulating software process models using large language model agents. *arXiv preprint arXiv:2403.15852* (2024).
- [30] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).
- [31] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [32] Shaoying Liu. 2004. *Formal engineering for industrial software development: Using the SOFL method*. Springer Science & Business Media.
- [33] Shaoying Liu. 2024. *Agile-SOFL*. Springer.
- [34] Shaoying Liu and Weikai Miao. 2021. A formal specification animation method for operation validation. *Journal of Systems and Software* 178 (2021), 110948.
- [35] Shaoying Liu and Shin Nakajima. 2020. Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing. *IEEE Transactions on Software Engineering* 48, 2 (2020), 691–712.
- [36] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. Specgen: Automated generation of formal program specifications via large language models. *arXiv preprint arXiv:2401.08807* (2024).
- [37] Cu D Nguyen, Alessandro Marchetto, and Paolo Tonella. 2013. Automated oracles: An empirical study on cost and effectiveness. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. 136–146.
- [38] A Jefferson Offutt, Yiwei Xiong, and Shaoying Liu. 1999. Criteria for generating specification-based tests. In *Proceedings Fifth IEEE International Conference on Engineering of Complex Computer Systems (ICECCS'99)(Cat. No. PR00434)*. IEEE, 119–129.
- [39] Ruwei Pan, Hongyu Zhang, and Chao Liu. 2025. CodeCoR: An LLM-based self-reflective multi-agent framework for code generation. *arXiv preprint arXiv:2501.07811* (2025).
- [40] Ruizhong Qiu, Weiliang Will Zeng, James Ezick, Christopher Lott, and Hanghang Tong. 2024. How efficient is llm-generated code? a rigorous & high-standard benchmark. *arXiv preprint arXiv:2406.06647* (2024).
- [41] Zeeshan Rasheed, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Codepori: Large scale model for autonomous software development by using multi-agents. (2024).
- [42] Debra Richardson, Owen O'Malley, and Cindy Tittle. 1989. Approaches to specification-based testing. In *Proceedings of the ACM SIGSOFT'89 third symposium on Software testing, analysis, and verification*. 86–96.
- [43] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive test generation using a large language model. *arXiv preprint arXiv:2302.06527* (2023).
- [44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [45] Colin Frank Snook. 2001. *Exploring the barriers to formal specification*. Ph. D. Dissertation. University of Southampton.
- [46] Phil Stocks and David Carrington. 2002. A framework for specification-based testing. *IEEE Transactions on software Engineering* 22, 11 (2002), 777–793.
- [47] Norbert Tihanyi, Yiannis Charalambous, Ridhi Jain, Mohamed Amine Ferrag, and Lucas C Cordeiro. 2025. A new era in software security: Towards self-healing software via large language models and formal verification. In *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 136–147.
- [48] Jianxun Wang and Yixiang Chen. 2023. A review on code generation with llms: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 284–289.
- [49] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2024. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439* (2024).
- [50] Zhitao Wang, Wei Wang, Zirao Li, Long Wang, Can Yi, Xinjie Xu, Luyang Cao, Hanjing Su, Shouzhi Chen, and Jun Zhou. 2024. Xuat-copilot: Multi-agent collaborative system for automated user acceptance testing with large language

model. *arXiv preprint arXiv:2401.02705* (2024).

[51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[52] Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H Chi, and Denny Zhou. 2023. Large language models as analogical reasoners. *arXiv preprint arXiv:2310.01714* (2023).

[53] Huan Zhang, Wei Cheng, Yuhan Wu, and Wei Hu. 2024. A pair programming framework for code generation via multi-plan exploration and feedback-driven refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1319–1331.

[54] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339* (2024).

[55] Ke Zhang, Chenxi Zhang, Chong Wang, Chi Zhang, YaChen Wu, Zhenchang Xing, Yang Liu, Qingshan Li, and Xin Peng. 2025. LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents. *arXiv preprint arXiv:2503.15079* (2025).

[56] Xinyue Zuo, Yifan Zhang, Hongshu Wang, Yufan Cai, Zhe Hou, Jing Sun, and Jin Song Dong. 2025. PAT-Agent: Autoformalization for Model Checking. *arXiv preprint arXiv:2509.23675* (2025).